



MyCodeSchool Teoria — Windows Forms

Ce ai nevoie sa stii pentru fiecare exercitiu · lectiile 1 si 2 · Stefan

Cum folosesti documentul: citesti sectiunea exercitiului, apoi il faci. Nu citi tot deodata — nu se retine nimic asa. Sectiunile sunt numerotate ca exercitiile din EXERCITII-01.pdf si EXERCITII-02.pdf.

Nu contine C#. Limbajul il stii. Contine doar ce e specific interfetelor grafice, plus mecanismul de dedesubt acolo unde el explica o capcana.

Ideea de baza — de ce se schimba totul

In consola, programul tau are un fir de executie si o ordine: `Main` porneste, codul curge in jos, un `while` asteapta la `Console.ReadLine()`. **Tu decizi ce se intampla dupa ce.**

Intr-o aplicatie cu ferestre, `Application.Run(new Form1())` porneste o **bucla de mesaje**: un `while` ascuns, scris de framework, care intreaba sistemul de operare „s-a intamplat ceva?” — o apasare de mouse, o tasta, o redimensionare. Cand da de ceva, cauta daca cineva a cerut sa fie anuntat si il cheama.

Consecinta practica: metodele tale nu se mai executa in ordinea in care le scrii. Ele **stau** pana cand bucla de mesaje le cheama. Nu mai exista „pasul urmator” — exista doar reactii. Si pentru ca bucla e **un singur fir**, cat timp metoda ta ruleaza, fereastra e inghetata: nu se redeseneaza, nu raspunde la click. De asta „aplicatia nu raspunde” e aproape mereu cod lung scris intr-un handler.

Lectia 1 — mediul, controalele, evenimentele

Cele trei fisiere si cuvantul `partial`

ex. 1

`Program.cs` contine `Main`. `Application.EnableVisualStyles()` spune ca butoanele sa arate ca in Windows-ul curent, nu ca in Windows 95. `Application.Run(...)` creeaza fereastra si porneste bucla de mesaje; cand fereastra se inchide, bucla se termina si `Main` se incheie.

Clasa `Form1` e declarata `partial`, adica **aceeasi clasa scrisa in doua fisiere**, pe care compilatorul le lipeste. In `Form1.cs` scrii tu; in `Form1.Designer.cs` scrie unealta de proiectare. E acelasi lucru ca la `Repository<T>`: o singura clasa, doar ca sursa ei e impartita.

De ce nu editezi `.Designer.cs`: nu e „interzis” ci inutil. Designer-ul **rescrie fisierul** din ce vede pe suprafata formei, de fiecare data cand muti un control. Orice ai scris de mana acolo dispare fara avertisment.

Proprietatile ferestrei

ex. 2

Proprietate	Ce face
<code>FormBorderStyle</code>	<code>FixedSingle</code> / <code>FixedDialog</code> = nu se redimensioneaza; <code>Sizable</code> = da
<code>StartPosition</code>	<code>CenterScreen</code> o pune pe mijloc; implicit e unde vrea Windows
<code>MaximizeBox</code> , <code>MinimizeBox</code>	butoanele din coltul din dreapta sus
<code>Text</code>	titlul din bara ferestrei — la un buton, eticheta de pe el

Toate sunt proprietati C# obisnuite. Fereastra **Properties** din Visual Studio nu face nimic magic: scrie atribuirii in `InitializeComponent()`.

Evenimentul e o lista de metode

ex. 3

Un buton are un camp `Click` de tip eveniment. Linia generata de Designer:

```
this.btnOk.Click += new EventHandler(this.btnOk_Click);
```

`EventHandler` e un **delegate**: un tip care descrie o semnatura, aici `void Metoda(object sender, EventArgs e)`. Operatorul `+=` **adauga** metoda ta intr-o lista pe care butonul o tine. Cand utilizatorul apasa, butonul parcurge lista si le cheama pe toate, in ordinea adaugarii.

De aici ies doua lucruri utile: un eveniment poate avea **mai multi** abonati (doua butoane pot folosi acelasi handler), iar `sender` iti spune **cine** l-a declansat — asa scrii un singur handler pentru zece butoane si te uiti in `sender` ca sa stii pe care s-a apasat.

Ordinea de tastatura

ex. 4

`TabIndex` e ordinea in care `Tab` plimba focusul. E numerotare per container: controalele dintr-un `GroupBox` se numereaza intre ele. `AcceptButton` pe forma = butonul apasat de `Enter`, `CancelButton` = cel apasat de `Esc`.

Nu e cosmetica: un formular pe care nu-l poti completa fara mouse e un formular incomplet, si se vede la punctajul de design.

Validare fara sa pierzi datele

ex. 5

`int.TryParse(text, out n)` returneaza `bool` si nu arunca exceptie. `int.Parse` arunca — si intr-o aplicatie cu ferestre o exceptie neprinsa inchide aplicatia cu datele utilizatorului in ea.

`textBox.Focus()` muta cursorul in campul gresit. `SelectAll()` ii selecteaza continutul, ca utilizatorul sa poata scrie peste.

Enabled si Visible

ex. 6

	Enabled = false	Visible = false
Se vede	da, gri	nu
Raspunde la mouse	nu	nu
Ocupa loc in layout	da	nu
Textul se poate copia	nu	—

Pentru text care trebuie citit si copiat, dar nu modificat, folosesti `ReadOnly = true` pe `TextBox`, nu `Enabled = false`.

Anchor si Dock

ex. 7

`Anchor` = **de care margini ale parintelui ramane legat controlul**. Implicit `Top`, `Left`: cand fereastra creste, controlul isi pastreaza distanta fata de sus si stanga, deci pare ca nu se misca. Daca pui `Top`, `Left`, `Right`, controlul isi pastreaza distanta fata de ambele laturi, deci **se intinde**. Ancorarea de doua laturi opuse = redimensionare; de o singura latura = deplasare.

`Dock` = controlul se lipeste de o latura si ocupa toata lungimea ei (`Fill` ocupa tot ce a ramas). Ordinea conteaza: fiecare control andocat consuma spatiu, iar urmatorul primeste ce a ramas.

Ce face Designer-ul, de fapt

ex. 8

```
Button btn = new Button();
btn.Text = "Apasa";
btn.Location = new Point(20, 30);
btn.Size = new Size(100, 30);
btn.Click += Btn_Click;
this.Controls.Add(btn);
```

Astea sase linii sunt exact ce scrie Designer-ul in `InitializeComponent()` cand tragi un buton pe forma. `Controls` e o colectie; un control exista pe ecran doar daca e in colectia `Controls` a unui parinte.

De ce conteaza la concurs: controalele care depind de date — zece butoane pentru zece categorii citite din fisier — nu pot fi desenate in Designer, pentru ca nu stii cate sunt pana nu rulezi. Le creezi in cod, in bucla.

Lectia 2 — liste si Toolbox

Firul rosu al lectiei: un control de lista are propria colectie, `Items`, care tine **ce afiseaza**. Datele tale stau in `List<T>`. Sunt doua colectii diferite. Bug-ul clasic de interfata e cand una se schimba si cealalta nu — pe ecran scrie altceva decat in memorie. Ai doua strategii: **le sincronizezi tu de mana** (ex. 9-12, 17-18) sau **legi controlul de lista ta** si framework-ul se ocupa (ex. 15, 19). A doua e mai putin cod si mai putine ocazii de greseala.

`ListBox.Items`

ex. 9

`Items.Add`, `Items.Remove`, `Items.Clear`, `Items.Insert`. `SelectedIndex` e pozitia selectata, sau `-1` daca nu e nimic selectat — de aici vine exceptia cand stergi fara selectie. `SelectedItem` e obiectul, sau `null`.

`Items.Remove(obiect)` sterge dupa valoare, `RemoveAt(index)` dupa pozitie. Pentru clase, egalitatea e pe referinta daca nu ai suprascris `Equals`.

Ce se afiseaza cand pui un obiect in lista

ex. 10

`Items` e o colectie de `object`. Cand trebuie sa deseneze un rand, controlul cheama `ToString()` pe fiecare element. De asta un obiect fara `ToString()` suprascris apare ca `Namespace.Produs`.

Avantajul fata de a adauga text: daca pui `"Lapte - 5 lei"`, ai pierdut obiectul — ca sa-l gasesti inapoi trebuie sa spargi sirul. Daca pui obiectul, `SelectedItem as Produs` ti-l da direct, cu toate campurile.

Doua liste

ex. 11

Nimic nou conceptual: scoti din una, adaugi in cealalta, si recalculezi `Enabled`-urile. Scrie **o singura metoda** care actualizeaza starea butoanelor si cheam-o dupa fiecare mutare — altfel ai patru locuri care trebuie tinute in acord, si unul va ramane in urma.

Selectie multipla si stergerea in bucla

ex. 12

`SelectionMode` : `One` , `MultiSimple` (click = bifa), `MultiExtended` (`Ctrl` si `Shift`). Selectia se citeste din `SelectedIndices` sau `SelectedItems` .

Capcana: cand stergi elementul de la pozitia `i` , tot ce urmeaza se muta cu o pozitie in stanga. Daca parcurgi crescator, urmatorul element ia locul celui sters si il sari. **Parcurgi invers**, de la sfarsit spre inceput — atunci stergerea afecteaza doar pozitii pe care le-ai vizitat deja.

DropDownStyle

ex. 13

Valoare	Lista	Se poate scrie?
<code>Simple</code>	mereu vizibila	da
<code>DropDown</code>	se deschide	da
<code>DropDownList</code>	se deschide	nu

Pentru un camp unde raspunsul trebuie sa fie din lista, `DropDownList` e singura alegere corecta: elimina din start orice validare a textului liber.

Liste legate

ex. 14

`SelectedIndexChanged` se declanseaza si cand schimbi selectia din cod, nu doar cand o schimba utilizatorul. Cand umpli al doilea combo, el isi seteaza singur o selectie si trage din nou evenimentul — de aici vin buclele de actualizari. Goleste intotdeauna inainte sa umpli.

DataSource si de ce ai nevoie de BindingList

ex. 15

Cand setezi `DataSource` , controlul nu mai foloseste `Items` — incearca sa adaugi acolo si primeste exceptie. `DisplayMember` e numele proprietatii afisate, `ValueMember` cea returnata de `SelectedValue` , `SelectedItem` ramane obiectul intreg.

Mecanismul: o `List<T>` nu are cum sa anunte pe nimeni ca s-a schimbat — nu are evenimente. Controlul citeste lista o data, la legare, si nu mai afla nimic. `BindingList<T>` implementeaza `IBindingList` si emite `ListChanged` la fiecare adaugare sau stergere; controlul e abonat si se redeseneaza singur. Aceeasi idee ca la `ToText` / `FromText` : informatia trebuie sa aiba pe unde sa circule, altfel se pierde.

CheckedListBox si evenimentul care se trage prea devreme

ex. 16

`CheckedItems` si `CheckedIndices` dau bifatele. `SetItemChecked(i, true)` bifeaza din cod.

Capcana: `ItemCheck` se declanseaza **inainte** ca starea sa se schimbe, si iti da noua valoare in `e.NewValue` . Daca numeri `CheckedItems.Count` acolo, numeri starea veche. Ori folosesti `e.NewValue` , ori amani numaratoarea.

ListView

ex. 17

Un rand e un `ListViewItem` . **Textul item-ului e prima coloana**; celelalte se adauga cu `SubItems.Add(...)` — de aceea nu adaugi prima valoare de doua ori. `View = Details` da tabelul, `FullRowSelect` selecteaza randul intreg, `GridLines` deseneaza liniile.

De ce sorteaza alfabetic

ex. 18

Tot ce intra in `ListView` devine `string`. Controlul nu stie ca „12” e numar, deci compara sirurile: „12” vine inaintea lui „9”, pentru ca „1” < „9”.

Sortarea se face cu `ListViewItemSorter`, caruia ii dai un obiect care implementeaza `IComparer`. In `Compare` primesti doi `ListViewItem`, iei `SubItems[coloana].Text`, il convertesti la numar cand coloana e numerica si compari. Semnul rezultatului decide ordinea; inversezi ordinea inmultind cu `-1`.

DataGridView legat

ex. 19

Cu `DataSource = bindingList`, coloanele se genereaza singure din proprietatile publice ale clasei. `AutoSizeColumnsMode = Fill` imparte latimea, `AllowUserToAddRows = false` scoate randul gol de la final, `SelectionMode = FullRowSelect` selecteaza randul.

Compara cu ex. 17: acolo ai scris bucla care umple tabelul, aici ai scris o atribuire.

Validarea in celula

ex. 20

`CellValidating` se declanseaza inainte ca valoarea sa fie acceptata si iti da `e.FormattedValue`. `e.Cancel = true` opreste iesirea din celula. `CellValueChanged` vine dupa, cand valoarea a intrat deja. `Rows[i].ErrorText` pune un semn rosu pe rand — mai putin intruziv decat un `MessageBox`.

TreeView

ex. 21

`Nodes` e colectia de noduri de nivel 1; fiecare `TreeNode` are la randul lui `Nodes`. `SelectedNode` da nodul curent, `Parent` da parintele (`null` la radacina), `Level` da adancimea.

Tag e o proprietate de tip `object` pe care o are aproape orice control. E locul unde legi elementul vizual de obiectul tau: `nod.Tag = categoria`, apoi `SelectedNode.Tag as Categorie`. Fara ea ai ramane sa deduci din text, care e fragil.

Meniuri si bare

ex. 22

`MenuStrip` = bara de meniu sus; elementele sunt `ToolStripMenuItem`, cu `ShortcutKeys` pentru scurtaturi. `ToolStrip` = bara cu butoane. `StatusStrip` = bara de jos, cu `ToolStripStatusLabel`. `ContextMenuStrip` se atribuie proprietatii `ContextMenuStrip` a unui control si apare la click dreapta.

Cand aceeasi actiune e si in meniu si pe bara, **scrie o singura metoda** si aboneaza ambele la ea. Doua copii ale aceleiasi logici se desincronizeaza.

Numere si progres

ex. 23

`NumericUpDown`: `Minimum`, `Maximum`, `Value` (de tip `decimal`, deci conversie explicita), `DecimalPlaces`. Avantajul fata de `TextBox`: nu se poate introduce text invalid, deci dispare o validare.

`TrackBar`: `Minimum`, `Maximum`, `Value`, evenimentul `Scroll`. `ProgressBar`: `Value` intre `Minimum` si `Maximum` — daca depasesti, arunca exceptie, deci limiteaza tu inainte.

Date si desenare proprie

ex. 24

`DateTimePicker.Value` e un `DateTime`. `Format` schimba afisarea; `MinDate` / `MaxDate` limiteaza alegerea.

Ca sa colorezi un rand din `ListBox`, pui `DrawMode = OwnerDrawFixed` si tratezi `DrawItem`: primesti un `Graphics` si dreptunghiul randului si desenezi tu fundalul si textul. Controlul nu mai deseneaza nimic singur — daca uiti sa desenezi textul, randul ramane gol.

Timer ruleaza pe firul ferestrei

ex. 25

`Interval` in milisecunde, `Start()`, `Stop()`, evenimentul `Tick`.

Foarte important: `System.Windows.Forms.Timer` **nu creeaza un fir nou**. Pune un mesaj in aceeaasi bucla de mesaje, deci `Tick` ruleaza pe firul interfetei. Avantaj: poti atinge controalele direct, fara complicatii. Dezavantaj: daca scrii ceva lung in `Tick`, fereastra ingheata, exact ca la orice alt handler. Si `Interval` e o promisiune, nu o garantie — daca firul e ocupat, tick-ul intarzie.

Containere

ex. 26

Fiecare container are propria colectie `Controls`, iar pozitiile copiilor sunt relative la el, nu la fereastra.

- `TabControl` — file; fiecare `TabPage` e un container separat
- `SplitContainer` — doua panouri cu o bara trasa de utilizator
- `TableLayoutPanel` — grila cu randuri si coloane in procente sau absolut
- `FlowLayoutPanel` — asaza copiii unul dupa altul si trece pe rand nou

Ultimele doua fac automat ce faceai manual cu `Anchor`, si rezista mai bine cand adaugi un control la mijloc.

Dialoguri

ex. 27

Toate se folosesc la fel:

```
OpenFileDialog dlg = new OpenFileDialog();
dlg.Filter = "Fisiere text|*.txt|Toate|*.*";
if (dlg.ShowDialog() == DialogResult.OK)
{
    ...dlg.FileName...
}
```

`ShowDialog()` e **modal**: blocheaza restul aplicatiei si nu returneaza pana nu se inchide fereastra. `Show()` e nemodal si returneaza imediat. Daca nu verifici rezultatul, continui cu valori vechi sau goale cand utilizatorul a apasat Cancel.

Text si imagini

ex. 28

`RichTextBox`: `SelectionFont` si `SelectionColor` se aplica pe selectia curenta; daca nu e nimic selectat, se aplica textului scris de acum inainte.

`MaskedTextBox.Mask`: `0` = cifra obligatorie, `9` = cifra optionala, `L` = litera obligatorie, `A` = alfanumeric.

`MaskCompleted` spune daca masca a fost completata integral.

`PictureBox.SizeMode`: `Zoom` incadreaza pastrand proportiile, `StretchImage` deformeaza, `Normal` taie.

Componentele care nu se vad

ex. 29

`ToolTip`, `ErrorProvider`, `Timer` si dialogurile nu se aseaza pe forma — apar intr-o tava sub suprafata de proiectare. Sunt **componente**, nu controale: nu au suprafata proprie, lucreaza asupra altor controale.

Un singur `ToolTip` serveste toata fereastra: `toolTip1.SetToolTip(btnSalveaza, "Salveaza produsul")`. La fel `ErrorProvider.SetError(txtEmail, "Email invalid")` pune semnul rosu, iar `SetError(txtEmail, "")` il scoate.

Diferenta fata de `MessageBox` : mesajul modal opreste lucrul si cere un click ca sa dispara.

`ErrorProvider` arata eroarea langa campul vinovat si lasa utilizatorul sa continue. La un formular cu trei greseli, primul il obliga sa le descopere una cate una.

Cand le pui la un loc

ex. 30

Nimic nou. Doua lucruri de urmarit:

- **O singura sursa a adevarului.** Cand aceleasi produse apar si in arbore, si in tabel, si in bara de stare, toate trei citesc din aceeasi `List<T>` ; nu tii trei copii.
- **O metoda de reimprospatare**, chemata dupa orice modificare, in loc sa actualizezi trei controale in cinci locuri diferite. E acelasi motiv pentru care ai scos cautarile in `Repository<T>` la academy.